HP-Shell Benchmarking Suite

Release 0.1

Elliot Melcer

04.06.2026

API-Referenz

Dokumentation der Benchmarking-Suite für HP-Schalen (Masterarbeit).

KAPITEL 1

analysis_core

Querschnitts- und Materialmethoden, Moment-Krümmungs-Analyse.

1.1 section_methods

Moment-curvature and cross-section methods per Eurocode 2.

Provides routines for computing the cracking moment, the SLS/ULS bending strength, and full as well as simplified moment-curvature (M-κ) diagrams, plus helpers for building SLS/ULS sections and querying section properties.

Units: lengths in mm, forces in N, moments in Nmm.

Author: Elliot Melcer

exception core.analysis_core.section_methods.**InvalidSectionForMKError**

Bases: ValueError

Raised when a section cannot produce a valid M-κ diagram.

Typical cause: prestress too high, so the section would crush before cracking.

core.analysis_core.section_methods.**calculate_cracking_moment_sls_Nmm_EC**(*section*, *n: float = 0.0*)

Calculate the cracking moment of a section at SLS.

Finds the strain profile where the bottom fiber reaches the cracking strain `eps_ctm = fctm / Ecm` while maintaining equilibrium with the applied axial force `n`. Includes physical strain-limit checks to avoid spurious solutions where the concrete would crush before cracking.

Parameter

- **section** (GenericSection) – Section to analyze (should be a ULS section; an SLS section is built internally).

- **n** (float, *optional*) – Applied axial force [N] (positive = tension, negative = compression). Default: 0.0.

Rückgabe

Result with the keys:

- section : GenericSection — the SLS section used.
- m_cr : float — cracking moment [Nmm], or float('-inf') if the section crushes before cracking.
- strain_profile : list — [eps_0, chi_y, chi_z] at cracking.
- valid : bool — True if a physically valid solution was found.
- reason : str or None — explanation if invalid, else None.
- chi_min_physical : float — present only on the invalid branch; the lower physical curvature bound [1/mm].
- eps_top : float — present only on the valid branch; top-fiber strain [-] of the solution.

Rückgabetyp

dict

Verursacht

ValueError – If no concrete geometry is found in the section.

Notes

Strain profile convention: eps(z) = eps_0 + chi_y * z. For sagging bending chi_y is negative, so the crushing limit acts as a lower bound on the curvature.

core.analysis_core.section_methods.**calculate_bending_strength_sls_Nmm_EC**(*section: GenericSection*, *n: float = 0.0*) → dict

Calculate the SLS bending strength of a section.

Builds an SLS section, computes the bending strength for the given axial force, and returns the result together with the associated strain profile.

Parameter

- **section** (GenericSection) – Section to analyze.
- **n** (float, *optional*) – Applied axial force [N] (positive = tension). Default: 0.0.

Rückgabe

Result with the keys:

- `section` : GenericSection — the SLS section used.
- `m_u` : float or None — SLS bending strength [Nmm], or `None` if the moment cannot be taken by the section.
- `strain_profile` : list or None — `[eps_0, chi_y, 0.0]` at ultimate, or `None` if invalid.
- `valid` : bool — `True` if a valid solution was found.
- `reason` : str or None — failure reason if invalid, else `None`.

Rückgabetyp
dict

Verursacht
ValueError – For any ValueError other than the section being unable to take the moment (such errors are treated as real bugs and re-raised).

core.analysis_core.section_methods.**calculate_bending_strength_uls_Nmm_EC**(*section: GenericSection*, *n: float = 0.0*) → dict

Calculate the ULS bending strength of a section.

Converts the input to a ULS section (so an SLS section may safely be passed), computes the bending strength for the given axial force, and returns the result together with the associated strain profile.

Parameter

- **section** (GenericSection) – Section to analyze (SLS or ULS; converted to ULS internally).
- **n** (float, *optional*) – Applied axial force [N] (positive = tension). Default: `0.0`.

Rückgabe

Result with the keys:

- `section` : GenericSection — the ULS section used.
- `m_u` : float or None — ULS bending strength [Nmm], or `None` if the moment cannot be taken by the section.
- `strain_profile` : list or None — `[eps_0, chi_y, 0.0]` at ultimate, or `None` if invalid.
- `valid` : bool — `True` if a valid solution was found.
- `reason` : str or None — failure reason if invalid, else `None`.

Rückgabetyp
dict

Verursacht
ValueError – For any ValueError other than the section being unable to take the moment (such errors are treated as real bugs and re-raised).

core.analysis_core.section_methods.**calculate_moment_curvature_sls_EC**(*section: GenericSection*, *n: float = 0.0*, *constitutive_law: str = 'TENSTIFF_PARABOLIC'*, *simplification: bool | int | float = False*, *debug: bool = False*) → MomentCurvatureResults

Compute the moment-curvature results for the given section.

Dispatches to the full or the simplified M-κ method depending on `simplification` and enforces a force-controlled (monotonically increasing) result. For prestressed sections an initial state point (κ_0, M=0) is added.

Parameter

- **section** (`GenericSection`) – Section to analyse (ULS).
- **n** (`float`, *optional*) – Axial force [N]. Default: `0.0`.
- **constitutive_law** (`str`, *optional*) – Keyword for the concrete constitutive law. Default: `"TENSTIFF_PARABOLIC"`.
- **simplification** (`bool` or `int` or `float`, *optional*) – Controls the simplified M-κ calculation in `_simplified_moment_curvature_method()`. `False` uses the full method; `True` or a positive number uses the simplified method. Default: `False`.
- **debug** (`bool`, *optional*) – Enables debug output. Default: `False`.

Rückgabe
The complete, force-controlled M-κ curve.

Rückgabetyp
`MomentCurvatureResults`

Verursacht
ValueError – If `simplification` is neither `False`, `True` nor a positive number.

core.analysis_core.section_methods.**calculate_prestress_forces_Nmm**(*section: GenericSection*) → tuple[float, float]

Calculate the prestressing moment and total prestress force.

For each prestressed reinforcement the prestressing force is `F_p = A_s * eps_ini * E_s` and the prestressing moment is `M_p = Σ(F_p * z_s)` about the section centroid.

Parameter
section (`GenericSection`) – SLS section with prestressed reinforcement.

Rückgabe
`(M_p, N_p)` where `M_p` is the prestressing moment [Nmm] (always positive) and `N_p` is the total prestress force [N].

Rückgabetyp
`tuple` of `float`

core.analysis_core.section_methods.**get_strain_at_point**(*strain_profile*, *y*, *z*) → float

Compute the strain at point (y, z) for a given strain profile.

Evaluates `eps_0 + chi_y * z + chi_z * y`.

Parameter

- **strain_profile** (`list`) – `[eps_0, chi_y, chi_z]` with axial strain [-] and curvatures [1/mm].
- **y** (`float`) – y-coordinate [mm].
- **z** (`float`) – z-coordinate [mm].

Rückgabe
Strain at point (y, z) [-].

Rückgabetyp
`float`

core.analysis_core.section_methods.**sls_section_EC**(*section: GenericSection*, *constitutive_law: str*) → GenericSection

Return the section with an SLS constitutive law for the concrete.

Parameter

- **section** (`GenericSection`) – Section to convert (SLS or ULS).
- **constitutive_law** (`str`) – Keyword for the constitutive law (see `create_sls_concrete_EC()` for available keywords).

Rückgabe
New section with the SLS concrete material; reinforcement unchanged.

Rückgabetyp
`GenericSection`

core.analysis_core.section_methods.**uls_section_EC**(*section: GenericSection*, *alpha_cc: float = 0.85*, *gamma_c: float = 1.5*) → GenericSection

Return the section with a ULS constitutive law (parabola-rectangle).

Parameter

- **section** (GenericSection) – Section to convert (SLS or ULS).
- **alpha_cc** (float, *optional*) – Effectiveness factor for the concrete compressive strength. Default: 0.85.
- **gamma_c** (float, *optional*) – Partial safety factor for concrete. Default: 1.5.

Rückgabe
New section with the ULS concrete material; reinforcement unchanged.

Rückgabetyp
GenericSection

core.analysis_core.section_methods.**flipped_section**(*section: GenericSection*) → GenericSection

Return the section rotated by 180°, for support bending strength.

Used when calculating the bending strength at a support, where the section is effectively flipped.

Parameter
section (GenericSection) – Section to flip.

Rückgabe
The section rotated 180° about its centroid, named "<name> (Support)".

Rückgabetyp
GenericSection

core.analysis_core.section_methods.**get_concrete**(*section: GenericSection*) → Concrete

Return the first concrete material found in the section geometry.

Parameter
section (GenericSection) – Section to query.

Rückgabe
The first concrete material in the geometry.

Rückgabetyp
Concrete

core.analysis_core.section_methods.**get_reinforcement**(*section: GenericSection*) → tuple[Reinforcement, float]

Return the first reinforcement material and its area.

Assumes all reinforcement diameters are the same.

Parameter
section (GenericSection) – Section to query.

Rückgabe
(reinforcement, area) where reinforcement is the first Reinforcement material and area is the corresponding bar area [mm^2].

Rückgabetyp
tuple

Verursacht

`ValueError` – If the geometry contains no reinforcement points, or no reinforcement material is found.

`core.analysis_core.section_methods.`**`get_number_of_reinforcements`**(*section: GenericSection*) → int

Count the reinforcement point geometries in the section geometry.

Parameter

`section` (`GenericSection`) – Section to query.

Rückgabe

Number of reinforcement point geometries.

Rückgabetyp

`int`

1.2 material_methods

`class` `core.analysis_core.material_methods.`**`CrackingConcreteLawEC`**(*strains: ndarray*, *stresses: ndarray*, *eps_cu1: float*, *eps_c1: float*, ***kwargs*)

Bases: `UserDefined`

Author: Elliot Melcer

A concrete constitutive law that handles tensile cracking correctly in moment-curvature analysis according to Eurocode.

The class overrides get_ultimate_strain of UserDefined to allow for cracking:

get_ultimate_strain(yielding)

Called by get_balanced_failure_strain() to determine chi_ultimate. Returns 1e6 as the positive (tensile) limit so that the concrete self-pairing in the double loop never produces the governing curvature. chi_ultimate is therefore always governed by the reinforcement-concrete pair (reinf fracture vs. concrete crushing), which is physically correct.

`get_ultimate_strain`(*yielding: bool = False*)

Return (compressive_limit, 1e6).

The 1e6 tensile side is intentionally unreachable so that the concrete self-pairing in get_balanced_failure_strain() never produces the minimum curvature. chi_ultimate ends up governed by the reinforcement-concrete pair instead.

class core.analysis_core.material_methods.**TensionStiffeningConcreteLawEC**(*strains: ndarray*, *stresses: ndarray*, *eps_cu1: float*, *eps_c1: float*, ***kwargs*)

Bases: *CrackingConcreteLawEC*

Author: Elliot Melcer A concrete constitutive law that handles tension stiffening

core.analysis_core.material_methods.**create_sls_concrete_EC**(*conc: Concrete | float | int*, *constitutive_law: str*) → Concrete

Creates an SLS Concrete object with new constitutive law according to Eurocode Available constitutive law keywords: NONE_PARABOLIC, FCTM_PARABOLIC, TENSTIFF_PARABOLIC, ELASTIC_ELASTIC :param conc: Concrete object or fck value :param constitutive_law: constitutive law (by keyword) :return:

core.analysis_core.material_methods.**create_uls_concrete_EC**(*conc: Concrete | float | int*, *alpha_cc: float = 0.85*, *gamma_c: float = 1.5*) → Concrete

Author: Elliot Melcer Creates a ULS Concrete object with parabola-rectangle constitutive law according to Eurocode :param conc: Concrete object or fck value :param alpha_cc: alpha_cc :param gamma_c: gamma_c :return:

core.analysis_core.material_methods.**fctm_parabolic_law_EC**(*concrete: Concrete | float | int*, *n_c: int = 80*, *n_t: int = 20*) → *CrackingConcreteLawEC*

Author: Elliot Melcer Creates a Non-Linear Constitutive Law with Linear Branch in Tension and Sargin Branch Under Compression - Tension: linear elastic (0 → eps_ctm) - Compression: Sargin (eps_cu1 → eps_c1)

Returns a CrackingConcreteLawEC Object

core.analysis_core.material_methods.**tenstiff_parabolic_law_EC**(*concrete: Concrete | float | int*, *n_c: int = 80*, *n_t: int = 20*) → *TensionStiffeningConcreteLawEC*

Author: Elliot Melcer Creates a Non-Linear Constitutive Law

Compression: Sargin Branch Tension: Linear Branch until fctm + Tension Stiffening according to Naya(2006)

Returns a TensionStiffeningConcreteLawEC Object

core.analysis_core.material_methods.**get_cube_EC**(*cylinder_strength*) → int

Author: Elliot Melcer Return cube strength (MPa) from EN 206 concrete class table.

core.analysis_core.material_methods.**get_cfrp_reinforcement_from_registry**(*mat_id: str*, *materials: dict*, *prestress_percent: float = 0.0*, *gamma_s: float = 1.3*)

Create a CFRP-Reinforcement object from materials registry.

Args:

mat_id: Material identifier (e.g., „solidian GRID Q142/142-CCE-25“) materials: Materials registry dictionary prestress_percent: Prestress level as percentage (0-100) gamma_s: Partial safety factor for reinforcement

Returns:

Reinforcement object

Raises:

KeyError: If mat_id not found in registry ValueError: If material type is not ‚reinforcement‘

Example:

materials = load_materials_registry(„materials.csv“) rebar = get_reinforcement_from_registry(

„solidian GRID Q142/142-CCE-25“, materials, prestress_percent=50.0

)

core.analysis_core.material_methods.**get_floor_material_from_registry**(*mat_id: str*, *materials: dict*)

Create a FloorMaterial object from materials registry.

Args:

mat_id: Material identifier materials: Materials registry dictionary

Returns:

FloorMaterial object

Raises:

KeyError: If mat_id not found in registry ValueError: If material type is not infill/insulation/screed

Example:
materials = load_materials_registry(„materials.csv“) infill = get_floor_material_from_registry(„generic_infill“, materials)

core.analysis_core.material_methods.**get_material_properties**(*mat_id: str*, *materials: dict*) → dict

Get raw material properties dictionary from registry.

Useful for accessing GWP, cost, and other properties not needed for structural analysis but needed for objective function.

Args:
mat_id: Material identifier materials: Materials registry dictionary

Returns:
Dictionary with material properties

Raises:
KeyError: If mat_id not found in registry

Example:
mat_props = get_material_properties(„solidian GRID Q142/142-CCE-25“, materials) gwp = mat_props[„gwp“] # kg CO2-eq cost = mat_props[„cost“] # €/unit

1.3 statics.deflection

Author: Elliot Melcer Deflection calculations

class core.analysis_core.statics.deflection.**DeflectionCalculator**

Bases: object

Calculate deflections using Simpson's rule and virtual work method. Uses position-dependent M-κ curves (calculated at support and midspan).

Position convention: x is normalized (0 at first support, 1 at second support, etc.)

static calculate_deflection_mm_EC(*slab_construction:* SlabConstruction, *loads: Loads*, *system: SystemType = SystemType.SIMPLE_BEAM*, *combination: str = 'QUASI_PERMANENT'*, *n_intervals: int = 40*, *N_axial_N: float = 0.0*, *constitutive_law: str = 'TENSTIFF_PARABOLIC'*, *load_history_method: str = 'NONE'*, *m_k_simplification=False*, *debug: bool = False*, *extended_debug: bool = False*) → float

Calculate maximum deflection using Simpson's rule and virtual work method. Uses position-dependent M-κ curves.

Parameter

- **load_history_method** – „NONE“ no load history considered, direct method used with parameter combination „FACTOR_EC“ load history considered according to Eurocode 2, Chapter 7.4.3, Equation (7.18) # „SECANT“ load history considered according to Kreller (1989)*, Chapter 4.2.4 # # *Title: „Zum nichtlinearen Trag- und Verformungsverhalten von Stahlbetonstabtragwerken unter Last- und Zwangeinwirkung“

- **m_k_simplification** – Control simplified M-K-Diagram Calculation For available inputs see _simplified_moment_curvature_method() in section_methods.py
- **constitutive_law** – Control Concrete Material Behavior in SLS for available inputs see sls_concrete() in material_methods.py
- **slab_construction** – Slab construction object
- **loads** – Loads object
- **system** – Structural system type
- **combination** – Load combination (Ignored when load_history_method='FACTOR_EC')
- **n_intervals** – Number of intervals for Simpson's rule (must be even)
- **N_axial_N** – Axial force [N] (positive = tension)
- **debug** – Enable debug output
- **extended_debug** – Enable extended debug output

Rückgabe

Maximum deflection [mm] Note: positive: sagging, negative: hogging

KAPITEL 2

slab_construction

Modellierung der Deckenkonstruktion und Plattentypen.

2.1 slab_construction

class slab_construction.slab_construction.**SlabConstruction**(*slab:* Slab, *floor:* Floor)

Bases: object

Author: Elliot Melcer Class to model a complete slab construction including load bearing structure and floor finishing

structural_dead_load_kN_m2() → float

Total structural dead load of the slab construction [kN/m^2].

non_structural_dead_load_kN_m2() → float

Total non-structural dead load of the slab construction [kN/m^2].

assert_infill_compatibility() → None

Asserts that the slab's infill material is the same object as the floor's infill layer material, if an infill layer is present in the floor.

Note: This is only relevant if slab construction is instantiated manually. One must make sure to use the same infill object for both the slab and the floor layer

Raises:

ValueError: If an infill layer exists in the floor but its material
is not the same object as the slab's infill material.

infill_area_density_kg_m2()

Returns the area density of the infill in [kg/m^2] Handles HPShell special case with minimum infill to achieve flat top

get_parameters() → dict

Returns a dictionary containing all slab construction parameters organized by category: Geometry, Concrete, and Reinforcement.

Note: This method assumes the slab is an HPSlab with an HPShell.

print_parameters() → None

2.2 floor

class slab_construction.floor.**FloorLayer**(*material: Material*, *thickness: float*)

Bases: object

Author: Elliot Melcer Basic Class to model floor layers Note: Thickness in [mm] to keep in line with all other dimensions in structuralcodes

material: Material

thickness: float

area_density_kg_m2()

Author: Elliot Melcer Returns area density for a FloorLayer of uniform thickness

class slab_construction.floor.**FloorMaterial**(*density: float*, *name: str | None = 'FloorMaterial'*)

Bases: Material

Author: Elliot Melcer Instantiable Simple Material for generic floor layers.

class slab_construction.floor.**ScreedMaterial**(*density: float*, *name: str | None = 'ScreedMaterial'*)

Bases: Material

Author: Elliot Melcer Instantiable Material for screed layers.

class slab_construction.floor.**InfillMaterial**(*density: float*, *name: str | None = 'InfillMaterial'*)

Bases: Material

Author: Elliot Melcer Instantiable Material for infill layers.

class slab_construction.floor.**InsulationMaterial**(*density: float*, *E_dyn: float*, *name: str | None = 'InsulationMaterial'*)

Bases: Material

Author: Elliot Melcer Instantiable Simple Material for insulation layers. Note: E_dyn in MN/m^3

class slab_construction.floor.**Floor**(*layers: list[*FloorLayer*] | None = None*)

Bases: object

Author: Elliot Melcer Class to model a floor construction made up of floor layers

layers: list[*FloorLayer*]

add_layer(*material: Material*, *thickness: float*) → None

Adds a layer to the floor construction. Note: Thickness in [mm] to keep in line with all other dimensions in structuralcodes Raises:

ValueError: If thickness is not positive. ValueError: If a layer with the same material type already exists in the floor.

dead_load() → float

Calculates the total dead load of the floor layers [kN/m^2].

get_layer_by_type(*material_type: type*) → *FloorLayer*

Returns the FloorLayer whose material is an instance of *material_type*.

Args:

material_type: A Material subclass, e.g. InsulationMaterial,
ScreedMaterial, InfillMaterial, FloorMaterial.

Raises:
KeyError: If no layer with the requested material type exists.

2.3 slabs.slab

class slab_construction.slabs.slab.**Slab**

Bases: ABC

Abstract base class for slab elements.

abstractmethod self_load() → float

Returns the self-weight load of the slab [kN/m^2]

abstractmethod infill_load() → float

Returns the load due to infill on the slab [kN/m^2]

2.4 slabs.one_way_slab

class slab_construction.slabs.one_way_slab.**OneWaySlab**

Bases: *Slab*, ABC

Author: Elliot Melcer Abstract Base Class that represents a one way slab Child Classes must implement L(), B() and section_at(x)

abstract property L: float

abstract property B: float

abstractmethod section_at(*x: float*) → GenericSection

This method returns the section of a one-way-slab at x. Note: Input domain must be normalized to the following model: # x = 0 at first support, x = 1 at second support, etc. # # **|-> x # 0 0.5 1 1.5 2 # # ==========|**... # /_ /_ /_# :param x: :return:

KAPITEL 3

ioh_core

Aufbau und Ausführung der IOH-Optimierungsprobleme.

3.1 problem_builder

Author: Max Dombrowski Modified by Elliot Melcer („Addition by: Elliot Melcer“ used to mark code)

- pass materials to analysis
- Let the slab module drop constraints that don't apply (i.e. hp_slab: defl_case_sls a/b cuts out unwanted checks from logging)

core.ioh_core.problem_builder.**build_problems_for_slab**(*slab_dir: Path*, *slab_module*) → dict[str, dict]

Build all IOH problems for a slab type directory and its analysis module.

- CSV-driven: parameter_defaults.csv, constraint_defaults.csv, problem_list.csv
- One EvalContext per problem (per-problem_ID), binding that problem's constraints into analysis()
- IOH constraints log *raw* violations from the cache (default: HIDDEN/1/1 unless overridden)

core.ioh_core.problem_builder.**build_problems_for_slab_type**(*slab_type: str*) → dict[str, dict]

Build all problems for a given slab type string.

Valid types are discovered dynamically by scanning ‚slab_benchmark.slabs‘ for subfolders with an ‚analysis.py‘. If the provided slab_type is invalid, a clear error lists all valid options.

3.2 cache_eval

Author: Max Dombrowski Modifications by Elliot Melcer:

- changed search strings to match analysis.py in hp_slab

class core.ioh_core.cache_eval.**EvalContext**(*decode_func*, *analysis_func*)

Bases: object

ensure_constraints(*names: list[str]*)

Declare stable attributes for all active constraints so IOH can watch them.

ensure_params(*var_names: list[str]*)

Declare stable attributes for decoded parameter values and their indices.

get(*x_idx*)

reset()

3.3 import_specs

Author: Max Dombrowski Modified by: Elliot Melcer

- main modified to work with file structure
- load_param_defaults: added lookup role
- loads constraint defaults when they are omitted from problem_list.csv

core.ioh_core.import_specs.**load_param_defaults**(*path: str | Path*) → Dict[str, dict]

Read parameter_defaults.csv and return:

{ name: {domain, values, lb, ub, fixed_idx, role, value_type} } or, for role='lookup': { name: {domain, values, role, value_type, lookup_key} }

Required columns:

name;domain;values;grid_start;grid_step;grid_count; default_lb;default_ub;default_fixed;role;value_type;looku

- **domain: ‚catalog' uses ‚values' as a |-separated list.**
  ‚grid' uses start/step/count to generate values.
- **role: ‚var' — optimization variable (lb/ub active)**
  ‚fixed' — constant per problem (fixed_idx active) ‚lookup' — derived from another parameter via lookup_key;

  never appears in problem_list.csv, resolved in decode()
- lb/ub/fixed_idx are 0-based indices into ‚values'.
- value_type: ‚float', ‚int', or ‚str' (default: ‚float' if empty).
- **lookup_key: only required for role='lookup'; must name another parameter**
  whose catalog/grid is positionally aligned with this one.

core.ioh_core.import_specs.**build_space**(*model: Dict[str, dict]*) → tuple[list[str], list[int], list[int]]

From a full parameter model (with roles), extract the optimization space: returns (var_names, lb_idx_list, ub_idx_list)

core.ioh_core.import_specs.**make_decode**(*model: Dict[str, dict]*, *var_names: List[str]*)

Build a decoder: decode(x_idx) -> dict of parameters, merging fixed parameters with variable indices (clamped to lb/ub). No aliases/synonyms. Parameter names must match the CSVs exactly.

core.ioh_core.import_specs.**load_constraint_defaults**(*path: str | Path*) → Dict[str, dict]

Read constraints_default.csv and return:
{ name: {kind, enforced, weight, exponent, active} } # modified by: Elliot Melcer

core.ioh_core.import_specs.**load_problems_combined**(*param_defaults: Dict[str, dict]*, *constr_defaults: Dict[str, dict]*, *problem_list_csv: str | Path*) → Dict[str, dict]

Read problem_list.csv and overlay per-problem configuration onto defaults. Returns:

{

problem_id: {
„model“: merged parameter model (roles + lb/ub/fixed_idx adjusted), „constraints“: { name: {kind, enforced, weight, exponent, active} }, „label“: str,

}

core.ioh_core.import_specs.**load_materials_registry**(*path: str | Path*) → Dict[str, dict]

Read materials.csv and return dictionary { name: {properties} }

Notes:

- ‚name‘ is the unique identifier (matches catalog values in parameter_defaults.csv)
- ‚type‘ indicates material category: reinforcement, infill, insulation, screed
- All numeric fields are converted to float (empty strings become None)
- Commas in numbers (European format) are converted to periods

3.4 io_util

Author: Max Dombrowski Modified by: Elliot Melcer

- main modified to work with file structure

3.5 experiment

core.ioh_core.experiment.**run_experiment**(*problem_bundle: dict[str, dict]*, *algorithm*, *n_runs: int*, *name: str = ''*, *zip_results: bool = True*, *delete_after_zip: bool = False*)

KAPITEL 4

Indizes

- genindex
- modindex
- search

Python-Modulindex

C

S